Messages & Conversational Chat History

Practical Learning Notes & Interview Preparation Guide

1. Concept Purpose

What problem does this concept solve? Large Language Models are completely **stateless**. They possess no native memory of previous turns. If a user asks "Who is the Prime Minister of the UK?" and follows up with "How old is he?", a stateless LLM cannot resolve "he" because each API request is completely isolated.

Why do we need it? To create conversational continuity, developers must collect and resend previous turns on every single request. LangChain provides structured message abstractions (`SystemMessage`, `HumanMessage`, `AIMessage`) and dynamic insertion placeholders (`MessagesPlaceholder`) to format conversational history accurately without manual string stitching.

Where is it used? Multi-turn chatbots, customer support copilots, conversational search, and agentic workflows where prior tool responses must be visible to subsequent reasoning steps.

Why did you use it in your implementation? In your repository (`chatbot/` and `Prompts/`), you implemented:

- Message arrays for basic role structuring (`chatbot/messages.py`).
- Tupled role templates with `ChatPromptTemplate` (`chatbot/chat_prompt_template.py`).
- An interactive multi-turn CLI chatbot loop (`chatbot/chatbot.py`).
- Dynamic history injection using `MessagesPlaceholder` (`Prompts/message_placeholder.py`).

2. Code Structure & Deep Analysis of Your Repository Files

File 1: `chatbot/messages.py`

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(
    model="openai/gpt-4o-mini",
    base_url="https://openrouter.ai/api/v1"
)

messages = [
    SystemMessage(content='You are a helpful assitant'),
    HumanMessage(content='Tell me about Langchain'),
]

result = model.invoke(messages)
messages.append(AIMessage(content=result.content))
print(messages)
```

Component Breakdown:

- `SystemMessage`: Sets the persona, behavior, guardrails, and role boundaries for the AI. Always placed first in the message array.

- `HumanMessage`: Represents input coming directly from the user.
- `AIMessage`: Represents responses generated by the model. Appending `result` directly (or constructing an `AIMessage` from `result.content`) records the assistant's own past dialogue for the next turn.

File 2: `chatbot/chat_prompt_template.py`

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage

chat_template = ChatPromptTemplate([
    ("system", "You are a helpful {domain} expert"),
    ("human", "Explain in simple words, what is {topic}")
])

prompt = chat_template.invoke({
    'domain': 'cricket',
    'topic': 'Bowling'
})
print(prompt)
```

Component Breakdown:

- `ChatPromptTemplate([...])`: Accepts a list of 2-tuples `("role", "template_string")`. LangChain recognizes `"system"`, `"human"`, `"user"`, `"ai"`, `"assistant"`.
- `chat_template.invoke({...})`: Replaces `{domain}` and `{topic}` with their concrete values and returns a `ChatPromptValue` holding typed message objects.

File 3: `chatbot/chatbot.py`

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

load_dotenv()

model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")

chat_history = [
    SystemMessage(content='You are a helpful assistant')
]

while True:
    user_input = input('you: ')
    if user_input.lower() == 'exit':
        break
    chat_history.append(HumanMessage(content=user_input))
    result = model.invoke(chat_history)
    chat_history.append(AIMessage(content=result.content))

    print("AI: ", result.content)
```

Component Breakdown:

- `chat_history`: An in-memory Python list initialized with the system prompt.
- `while True`: The interactive terminal loop. On each iteration:
  1. Reads user input and appends a new `HumanMessage`.
  2. Passes the entire growing `chat_history` list to `model.invoke(chat_history)`.

3. Appends the model's `AIMessage`.

File 4: `Prompts/message_placeholder.py`

```
import ast
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

chat_template = ChatPromptTemplate([
    ('system', 'you are a helpful customer support agent'),
    MessagesPlaceholder(variable_name='chat_history'),
    ('human', '{query}')
])

chat_history = []
with open('Prompts/chat_history.txt') as f:
    for line in f:
        message_type, _, content = line.partition('(content=')
        if content:
            role = 'human' if message_type.startswith('HumanMessage') else 'ai'
            chat_history.append((role, ast.literal_eval(content.rstrip().removesuffix(')'))))

prompt = chat_template.invoke({
    'chat_history': chat_history,
    'query': 'Where is my refund'
})
print(prompt)
```

Component Breakdown:

- `MessagesPlaceholder(variable_name='chat_history')`: Designates a slot where a list of messages will be inserted. Unlike string variables (`{query}`) which inject raw text, `MessagesPlaceholder` unpacks an entire list of message objects directly into the message array.

3. Internal Execution Flow

When multi-turn history is sent to a model:

1. **Placeholder Unpacking:** `ChatPromptTemplate.invoke()` processes each element in its template list. When it encounters `MessagesPlaceholder(variable_name='chat_history')`, it looks up `'chat_history'` in the passed dictionary, verifies it is a sequence of messages, and flattens them into the sequence.
2. **Token Accumulation:** On Turn 1, the model consumes System + User tokens. On Turn 2, the model consumes System + User1 + AI1 + User2 tokens. Total prompt tokens grow quadratically ($O(N^2)$ across N conversation turns).
3. **Wire Format Conversion:** LangChain converts the list of messages into provider JSON:

   ```
   {
     "messages": [
       {"role": "system", "content": "You are a helpful assistant"},
       {"role": "user", "content": "My name is Harsh"},
       {"role": "assistant", "content": "Hello Harsh! How can I help you today?"},
       {"role": "user", "content": "What is my name?"}
     ]
   }
   ```

4. **Attention Mechanism:** The transformer processes all previous tokens through causal self-attention, allowing the model to attend to the prior turn where the user stated their name.

4. Workflow Diagram

```
+------------------+ +-------------------------------+ | SystemMessage | --> | "Helpful Customer
Assistant" | +------------------+ +-------------------------------+ | +------------------+ | |
chat_history.txt | v | (Past turns) | --> +-------------------------------+ +------------------+ |
MessagesPlaceholder | | [Human: Order #12345 refund?] | | [AI: Initiated in 3-5 days] |
+-------------------------------+ | +------------------+ | | Current Query | -->
+-------------------------------+ | {query} | | HumanMessage: "Where is it?" | +------------------+
+-------------------------------+ | v Compiled ChatPromptValue +-------------------------------+ |
BaseChatModel (OpenAI gpt-4o) | +-------------------------------+ | v +-------------------------------+
| AIMessage: "Regarding order | | #12345, it is in transit..." | +-------------------------------+
```

5. Recommended Production Architecture (Session Persistence)

In-memory lists (`chat_history = []`) fail in production because:

1. If the server restarts or crashes, all conversations are wiped.
2. Multiple concurrent users share the same global list or overwrite each other.
3. Lists grow indefinitely, eventually exceeding the model's maximum context window length.

The enterprise pattern uses a session-backed storage mechanism (Redis, SQLite, or PostgreSQL) keyed by `session_id`:

```
production_chatbot/
|
├── config/
|   └── settings.py
|
├── storage/
|   ├── __init__.py
|   └── chat_history_repo.py  # SQLite / Redis persistence
|
├── services/
|   ├── __init__.py
|   └── conversation_service.py # Session isolation & history truncation
|
└── main.py
```

6. Implementation From Scratch (Step-by-Step Exercise)

Objective: Build a production-grade Persistent Chatbot using SQLite to save conversation history per `session_id`, with a 4-message sliding window to prevent context overflow.

```
# Reference Solution using SQLite
import sqlite3
import json
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage, messages_from_dict, messa

class SQLiteChatHistory:
    def __init__(self, db_path="chat_history.db"):
        self.conn = sqlite3.connect(db_path, check_same_thread=False)
        self.conn.execute('''CREATE TABLE IF NOT EXISTS history (session_id TEXT PRIMARY KEY, message
        self.conn.commit()

    def get_messages(self, session_id: str, limit: int = 4):
```

```
        cursor = self.conn.cursor()
        cursor.execute("SELECT messages_json FROM history WHERE session_id = ?", (session_id,))
        row = cursor.fetchone()
        if not row:
            return []
        all_msgs = messages_from_dict(json.loads(row[0]))
        return all_msgs[-limit:]

    def add_messages(self, session_id: str, new_messages: list):
        cursor = self.conn.cursor()
        cursor.execute("SELECT messages_json FROM history WHERE session_id = ?", (session_id,))
        row = cursor.fetchone()
        existing = json.loads(row[0]) if row else []
        existing.extend(messages_to_dict(new_messages))
        cursor.execute('''INSERT INTO history (session_id, messages_json) VALUES (?, ?)
            ON CONFLICT(session_id) DO UPDATE SET messages_json = excluded.messages_json''', (session_
        self.conn.commit()

# Interactive Run
model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")
storage = SQLiteChatHistory()
session_id = "user_harsh_001"

system_prompt = SystemMessage(content="You are a polite financial advisor.")

while True:
    user_text = input("User: ")
    if user_text.lower() == "exit":
        break

    # Retrieve last 4 historical messages
    past_history = storage.get_messages(session_id, limit=4)

    # Assemble full conversation payload
    current_human = HumanMessage(content=user_text)
    full_payload = [system_prompt] + past_history + [current_human]

    response = model.invoke(full_payload)
    print("AI:", response.content)

    # Persist the new turn
    storage.add_messages(session_id, [current_human, response])
```

7. Common Errors & Actual Repository Bugs

ERROR 1: Fragile String Parsing with ast.literal_eval (From Prompts/message_placeholder.py)

Your Code:

```
message_type, _, content = line.partition('(content=')
chat_history.append((role, ast.literal_eval(content.rstrip().removesuffix(')'))))
```

Why It Fails: You stored message string representations (`HumanMessage(...)`) in a plain text file and parsed them using string partitioning and `ast.literal_eval`. If an LLM response contains parentheses `)`, unescaped quotes, or linebreaks, `ast.literal_eval` throws a `SyntaxError` or truncates the content.

LangChain Standard Solution: Always use `messages_to_dict` and `messages_from_dict` from `langchain_core.messages` to serialize message lists to standard JSON:

```
from langchain_core.messages import messages_to_dict, messages_from_dict
import json

# To save:
Path("history.json").write_text(json.dumps(messages_to_dict(chat_history)))

# To load:
loaded_history = messages_from_dict(json.loads(Path("history.json").read_text()))
```

ERROR 2: Unbounded Context Window Memory Leak

In `chatbot/chatbot.py`, `chat_history.append(...)` runs indefinitely inside `while True`. Every LLM has a finite context window limit (e.g. 128k tokens). Once `chat_history` exceeds this limit, the API call crashes with a `ContextWindowExceededError`. Always implement sliding-window truncation (e.g. `chat_history[-10:]`) or summarization memory.

8. Explain It to Someone Else (Speaking Frameworks)

- **Simple Explanation:** "Since the AI forgets everything the moment it answers, chat history is like printing out the entire chat transcript and handing it back to the AI every time you send a new message."
- **Technical Explanation:** "Conversational memory in LangChain is maintained by serializing message history into a sequence of polymorphic `BaseMessage` objects. `MessagesPlaceholder` dynamically injects this sequence into a `ChatPromptTemplate` at invocation time, compiling a stateful message array for stateless LLM inference."
- **60-Second Interview Explanation:** "LLMs are completely stateless REST endpoints. To build a conversational chatbot, we must manage dialogue state on the client or application layer. In LangChain, I utilize `ChatPromptTemplate` combined with `MessagesPlaceholder`. This allows us to inject previous message turns dynamically between the system persona and current human query. In production, I back this with persistent storage (like SQLite or Redis) keyed by session IDs and enforce windowing or token pruning to prevent context overflow."
- **Whiteboard Explanation:** Draw a timeline with Turn 1, Turn 2, Turn 3. Under each turn, show the payload size doubling: `[System, U1, A1, U2, A2, U3]`. Highlight the need for a sliding window or summarizer node before calling `LLM.invoke()`.
- **Real-World Analogy:** An email thread. When you reply to an email, your email client automatically quotes all previous replies at the bottom so the recipient has the full context.

9. Interview Preparation Questions & Answers

Level	Question	Technically Exact Answer
Basic	What are the three core message types in LangChain?	`SystemMessage` (sets model persona and behavioral constraints), `HumanMessage` (inputs submitted by the user), and `AIMessage` (outputs generated by the model, including text content and tool call requests).
Basic	What is the purpose of `MessagesPlaceholder`?	It acts as a dynamic wildcard slot inside a `ChatPromptTemplate`. Unlike standard string placeholders that inject a single formatted string, `MessagesPlaceholder` unpacks an arbitrary list of `BaseMessage` objects into the prompt.

Intermediate	Why is maintaining conversation history in a global Python list dangerous?	1. Memory leaks and concurrency collisions (multi-user requests will overwrite or see each other's data). 2. Volatility (all state is lost on process restart). 3. Context window overflow (unbounded lists eventually exceed model token limits).
Intermediate	How do you prune conversation history when it gets too long?	Two primary strategies: 1. Sliding Window (retaining only the most recent k messages, e.g. `messages[-10:]`). 2. Summarization (using an LLM background chain to summarize older conversation turns into a concise `SystemMessage`).
Advanced	How does LangChain serialize messages to disk safely?	Via `messages_to_dict` and `messages_from_dict` from `langchain_core.messages`, or `langchain_core.load.dumps`/`loads`. This preserves message classes, role types, additional kwargs, tool calls, and IDs in standard JSON without fragile string parsing.
Advanced	What is `ToolMessage` and where does it fit in the message hierarchy?	`ToolMessage` represents the output of an executed tool/function. When an LLM returns an `AIMessage` with `tool_calls`, the application must execute the tool and append a `ToolMessage(content=..., tool_call_id=...)` before invoking the model again.